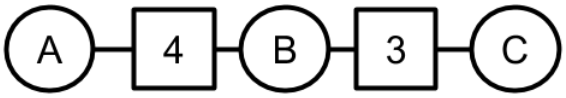
Q5. [18 pts] Dudoku

Here we introduce Dudokus, a type of CSP problem. A Dudoku consists of variables and summation constraints. The circles indicate variables that can take integer values in a specified range and the boxes indicate summation constraints, which specify that the variables connected to the constraint need to add up to the number given in the box.

(a) Let's begin with linear Dudokus, where the variables can be arranged in a linear chain with constraints between adjacent pairs. For example, in the linear Dudoku below, variables A, B, and C need values assigned to them in the set $\{1, 2, 3\}$ such that $A + B = 4$ and $B + C = 3$.



(i) [2 pts] How many solutions does this Dudoku have?

◯ 0 ◯ 1 ◯ 2 ◯ 3 ◯ more than 3

(ii) [1 pt] Consider the general case of a linear Dudoku with n variables $X_1, \ldots, X_n$, each taking a value in $\{1, \ldots, d\}$. What is the complexity for solving such a Dudoku using the generic tree-CSP algorithm?

◯ $\mathcal{O}(nd^3)$ ◯ $\mathcal{O}(n^2d^2)$ ◯ $\mathcal{O}(nd^2)$ ◯ $\mathcal{O}(d^n)$

(iii) [2 pts] One proposal for solving linear Dudokus is as follows: for each possible value i of the first variable X_1 in the chain, instantiate X_1 with that value and then run generic arc consistency beginning with the pair (X_2, X_1) until termination; keep going until a solution is found or there are no more values to try for X_1. Which of the following are true?

☐ This will correctly detect any unsolvable Dudoku.

☐ This will always solve any solvable Dudoku.

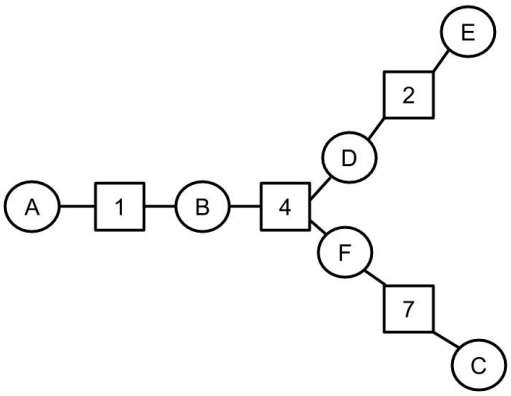
☐ This will sometimes terminate without finding a solution when one exists.

☐ The runtime is $O(nd^3)$.

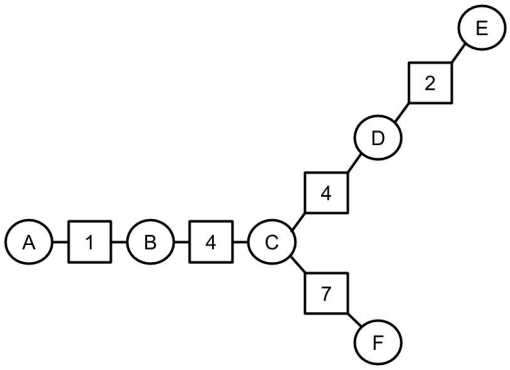
(iv) [2 pts] Binary Dudoku constraints are *one-to-one*, meaning that if one variable in a binary constraint has a known value, there is only one possible value for the other variable. Suppose we modify arc consistency to take advantage of one-to-one constraints instead of checking all possible pairs of values when checking a constraint. Now the runtime of the algorithm in the previous part becomes:

◯ $\mathcal{O}(nd)$ ◯ $\mathcal{O}(nd^3)$ ◯ $\mathcal{O}(n^2d^2)$ ◯ $\mathcal{O}(nd^2)$

(b) [4 pts] Branching Dudokus



Example A

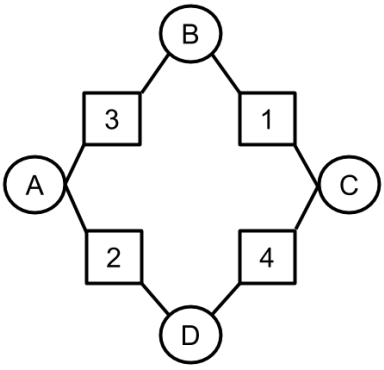


Example B

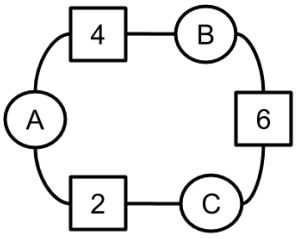
A branching Dudoku is one in which multiple linear chains are joined together. Chains can be joined at a summation node, as in example A above, or at a variable, as in example B above. Recall that a cutset is a set of nodes that can be removed from a graph to ensure some desired property. Which of the following are true?

- ☐ Dudoku A is a binary CSP.
- ☑ Dudoku B is a binary CSP.
- ☐ Dudoku A is a tree-structured CSP.
- ☑ Dudoku B is a tree-structured CSP.
- ☑ If variables B, D, and F are merged into a single megavariable, Dudoku A will be a tree-structured CSP.
- ☐ If variables A and E are merged into a single megavariable, Dudoku B will be a tree-structured CSP.
- ☐ The minimum cutset that turns Dudoku A into a set of disjoint linear Dudokus contains 3 variables.
- ☑ The minimum cutset that turns Dudoku B into a set of disjoint linear Dudokus contains 1 variable.

(c) Circular Dudokus



Example 1



Example 2

(i) [2 pts] The figure above shows two examples of circular Dudokus. If we apply cutset conditioning with a minimal cutset and a tree-CSP solver, what is the runtime for a circular Dudoku with n variables and domain size d?

○ $\mathcal{O}(d^{n-1})$ ○ $\mathcal{O}(n^2d^2)$ ○ $\mathcal{O}(nd)$ ✓ $\mathcal{O}(nd^3)$

(ii) [2 pts] Suppose that the variables in the circular Dudokus in the figure are assigned values such that all the constraints are satisfied. Assume also that the variable domains are integers in $[-\infty, \infty]$. Now consider what happens if we add 1 to the value of variable A in each Dudoku and then re-solve with A fixed at its new value. Which of the following are true?

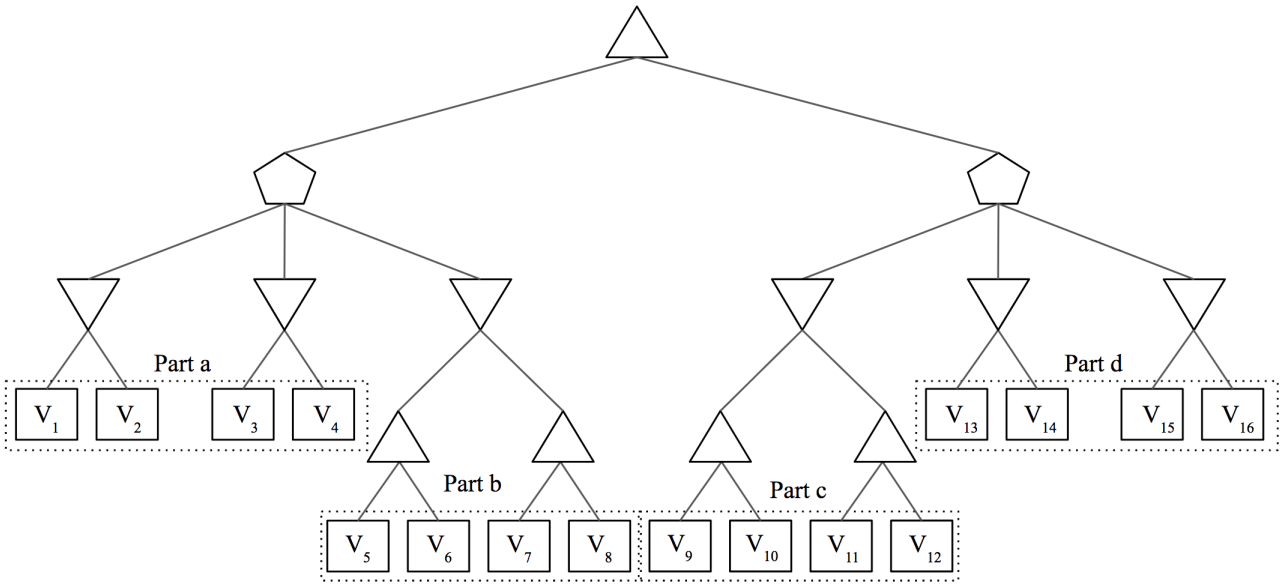
☐ Dudoku 1 now has no solution.

☑ Dudoku 2 now has no solution.

(iii) [4 pts] What can you conclude about the number of solutions to a circular Dudoku with n variables?

Q7. [16 pts] MedianMiniMax

You're living in utopia! Despite living in utopia, you still believe that you need to maximize your utility in life, other people want to minimize your utility, and the world is a 0 sum game. But because you live in utopia, a benevolent social planner occasionally steps in and chooses an option that is a compromise. Essentially, the social planner (represented as the pentagon) is a median node that chooses the successor with median utility. Your struggle with your fellow citizens can be modelled as follows:



There are some nodes that we are sometimes able to prune. In each part, mark all of the terminal nodes such that **there exists a possible situation** for which the node **can be pruned**. In other words, you must consider **all** possible pruning situations. Assume that evaluation order is **left to right** and all V_i's are **distinct.**

Note that as long as there exists ANY pruning situation (does not have to be the same situation for every node), you should mark the node as prunable. Also, alpha-beta pruning does not apply here, simply prune a sub-tree when you can reason that its value will not affect your final utility.

(a)
- ☐ V_1
- ☐ V_2
- ☐ V_3
- ☐ V_4
- ■ None

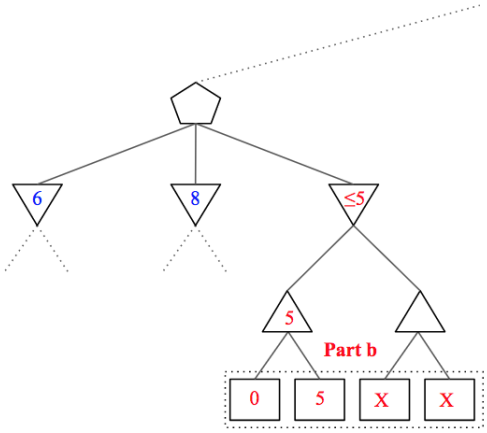
(b)
- ☐ V_5
- ■ V_6
- ■ V_7
- ■ V_8
- ☐ None

(c)
- ☐ V_9
- ☐ V_{10}
- ■ V_{11}
- ■ V_{12}
- ☐ None

(d)
- ☐ V_{13}
- ■ V_{14}
- ■ V_{15}
- ■ V_{16}
- ☐ None

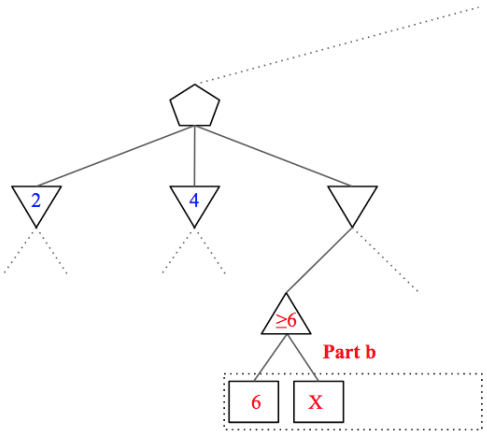
SID: _____

Part a:
For the left median node with three children, at least two of the childrens' values must be known since one of them will be guaranteed to be the value of the median node passed up to the final maximizer. For this reason, none of the nodes in part a can be pruned.



The value of this subtree will only get smaller.

The median node will **NOT** choose the value of this subtree. 6 is the median.



The value of this subtree will only get bigger.

*If the value of this subtree is chosen by the minimizer**, it will **NOT** be chosen by the median node.

**It is possible that the median is the value of the subtree to the right that we haven't looked at yet*

Part b (pruning V_7, V_8):
Let min_1, min_2, min_3 be the values of the three minimizer nodes in this subtree.

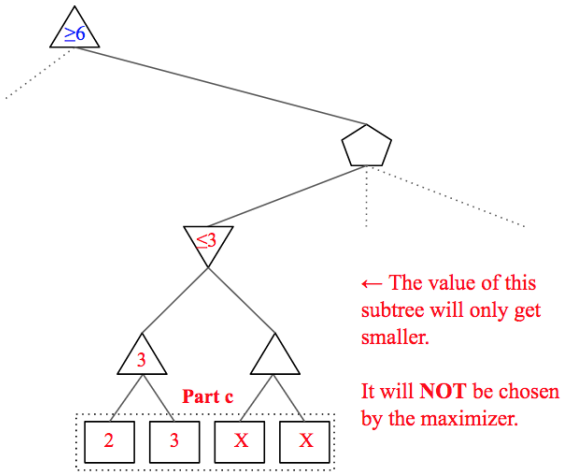
In this case, we may not need to know the final value min_3. The reason for this is that we may be able to put a bound on its value after exploring only partially, and determine the value of the median node as either min_1 or min_2 if $min_3 \leq \min(min_1, min_2)$ or $min_3 \geq \max(min_1, min_2)$.

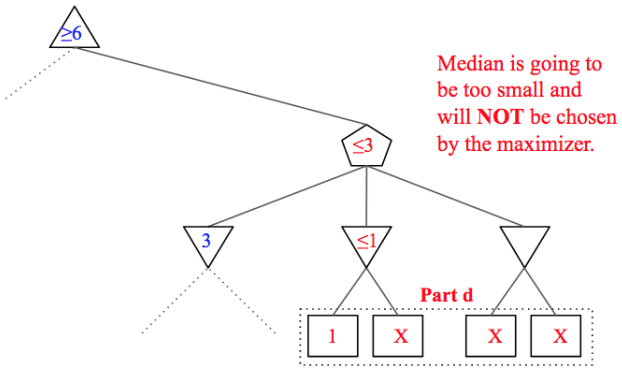
We can put an upper bound on min_3 by exploring the left subtree V_5, V_6 and if $\max(V_5, V_6)$ is lower than both min_1 and min_2, the median node's value is set as the smaller of min_1, min_2 and we don't have to explore V_7, V_8 in Figure 1.

Part b (pruning V_6):
It's possible for us to put a lower bound on min_3. If V_5 is larger than both min_1 and min_2, we do not need to explore V_6.

The reason for this is subtle, but if the minimizer chooses the left subtree, we know that $min_3 \geq V_5 \geq \max(min_1, min_2)$ and we don't need V_6 to get the correct value for the median node which will be the larger of min_1, min_2.

If the minimizer chooses the value of the right subtree, the value at V_6 is unnecessary again since the minimizer never chose its subtree.





Part c (pruning V_{11}, V_{12}):
Assume the highest maximizer node has a current value $max_1 \geq Z$ set by the left subtree and the three minimizers on this right subtree have value min_1, min_2, min_3.

In this part, if $min_1 \leq \max(V_9, V_{10}) \leq Z$, we do not have to explore V_{11}, V_{12}. Once again, the reasoning is subtle, but we can now realize if either $min_2 \leq Z$ or $min_3 \leq Z$ then the value of the right median node is for sure $\leq Z$ and is useless.

Only if both $min_2, min_3 \geq Z$ will the whole right subtree have an effect on the highest maximizer, but in this case the exact value of min_1 is not needed, just the information that it is $\leq Z$. Clearly in both cases, V_{11}, V_{12} are not needed since an exact value of min_1 is not needed.

We will also take the time to note that if $V_9 \geq Z$ we do have to continue the exploring as V_{10} could be even greater and the final value of the top maximizer, so V_{10} can't really be pruned.

Part d (pruning V_{14}, V_{15}, V_{16}):
Continuing from part c, if we find that $min_1 \leq Z$ and $min_2 \leq Z$ we can stop.

We can realize this as soon we explore V_{13}. Once we figure this out, we know that our median node's value must be one of these two values, and neither will replace Z so we can stop.

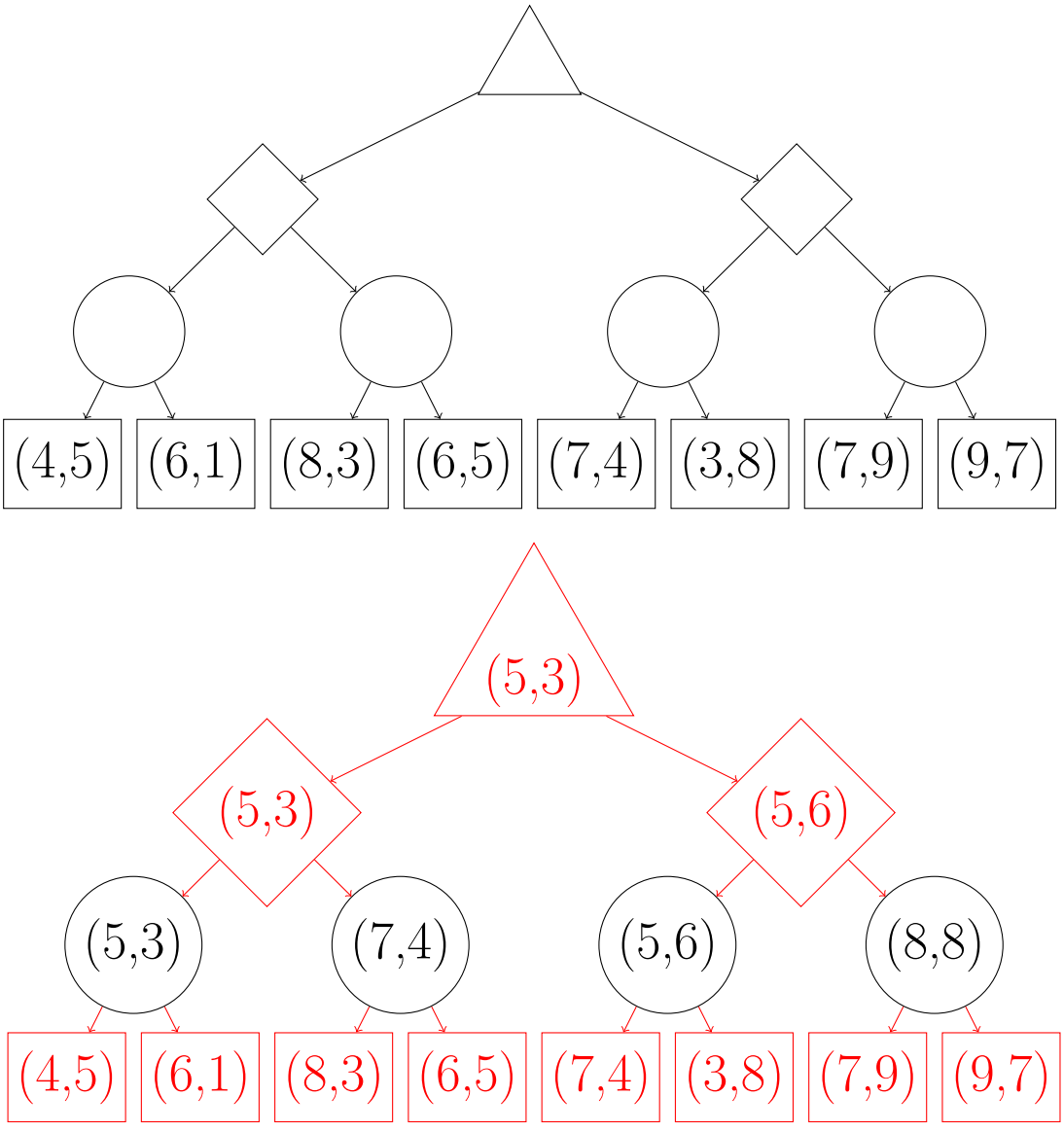
Q5. [16 pts] Trees

(a) [4 pts]
Consider the following game tree for a two-player game. Each leaf node assigns a score to each player, in order (e.g. the leaf node (3, 1) corresponds to player 1 getting 3 points and player 2 getting 1 point).

Note that while player 1's goal is to maximize her own score, player 2's goal is to maximize his relative score (i.e. maximize the expression "player 2's score minus player 1's score").

An upward-facing triangle represents a node for player 1, a diamond represents a node for player 2, and a circle represents a chance node (where either branch could be chosen with equal probability).

Fill in the values of all of the nodes; in addition, put an X on the line of all branches that can be pruned, or write "No pruning possible" below the tree if this is the case. Assume that branches are explored left-to-right.



No pruning.

(b) [4 pts]

Based on the above strategies for each player, fill in the two sides of the inequality below with expressions using the below variables so that if the following inequality holds true, we can prune for player 2. If no pruning is ever possible in a search tree for this game, write 'No pruning possible.'

You may make reference to the following variables:

- α_1 and α_2 are the scores for the best option that any node for player 1 on the path from the root to the current node has seen, including the current node.
- β_1 and β_2 are the scores for the best option that any node for player 2 on the path from the root to the current node has seen, including the current node.

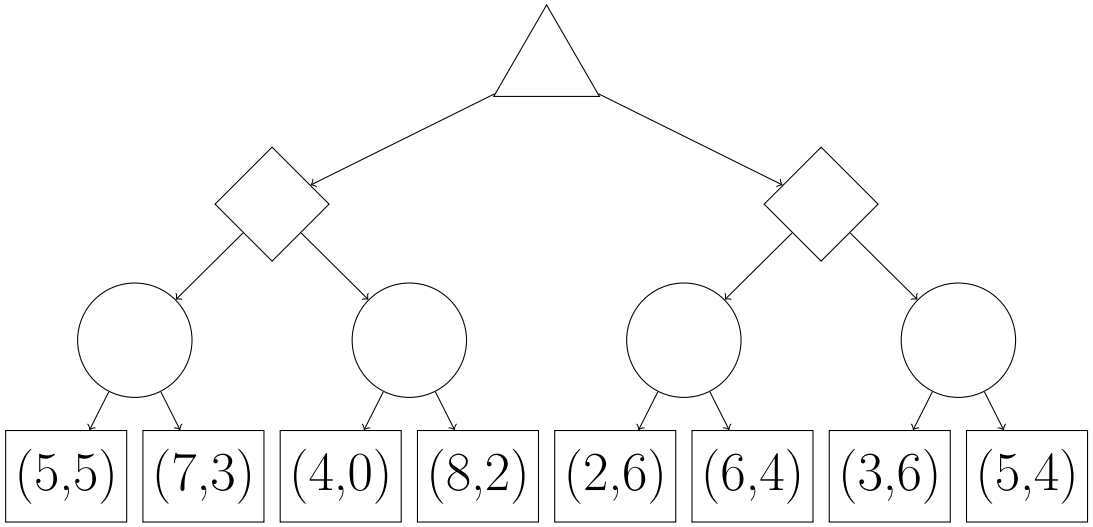
_____ > _____

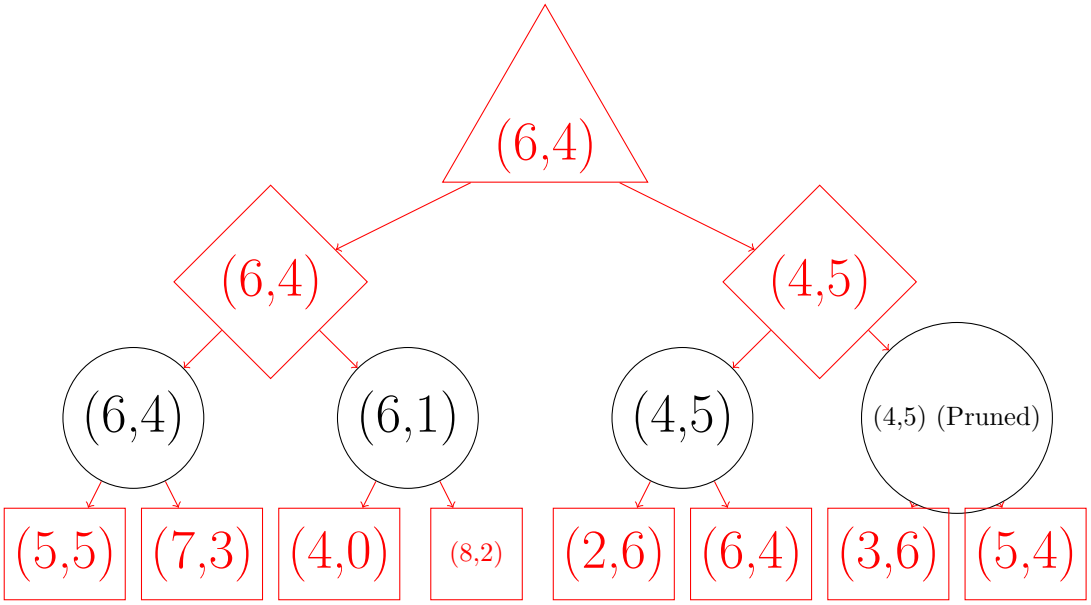
No pruning is possible. For each of the leaf states, we can replace player 2's score with the utility player 2 associates with that state (i.e. subtract player 1's score), and the resulting tree is a regular two-player non-zero-sum game, which cannot be pruned

(c) [4 pts]

Player 1's and player 2's goals remain the same from the two previous parts. However, in this game, no pair of scores at a leaf node can have a sum greater than 10, and both players know this.

Fill in the values of all of the nodes, and put an X on the line of all branches that can be pruned, or write 'No pruning possible' below the tree if this is the case. Assume that branches are explored left-to-right.





(d) [4 pts]

Taking into account this new information, fill in the two sides of the inequality below with expressions using the below variables so that if the following inequality holds true, we can prune for player 2. If no pruning is ever possible in a search tree for this game, write 'No pruning possible.'

You may make reference to the following variables:

- α_1 and α_2 are the scores for the best option that any node for player 1 on the path from the root to the current node has seen, including the current node.
- β_1 and β_2 are the scores for the best option that any node for player 2 on the path from the root to the current node has seen, including the current node.

_____ > _____

Any option that player 2 would prefer to (β_1, β_2) must have player 2's score increase by more than player 1's score. In total, both players' scores can increase by at most $10 - \beta_1 - \beta_2$ points, so the highest score player 1 can get in any option player 2 chooses would be $\beta_1 + (10 - \beta_1 - \beta_2)/2$. Pruning can occur if this number is still less than the best option that player 1 has seen for herself, α_1. Thus, the pruning condition is $\alpha_1 > (10 + \beta_1 - \beta_2)/2$.